

R T O S 项目多任务设计教程

生成日期：2 0 2 6 - 0 5 - 1 2

R T O S 项目多任务设计教程：任务、事件、队列、状态机到底怎么搭

引言：* * 适合对象 * *：刚开始做 R T O S 项目的嵌入式工程师，或者已经写过任务、队列，但对系统架构还不够清晰的开发者。

引言：* * 核心目标 * *：不是背概念，而是在新项目开发前，能判断什么时候该开任务、什么时候该用队列、什么时候只是事件、什么时候必须上状态机。

0 . 先用一个开锁流程建立直觉

假设设备支持云端开锁：

```
云端下发开锁命令
MQTT 收到命令
RS485 发送开锁帧
车辆返回 ACK
3 秒没回 ACK 就失败
成功或失败都要上报云端
```

很多项目一开始会写成：

```
if (recv_unlock_cmd)
{
    wakeup_device();
    rs485_send_unlock();

    while (!ack)
    {
        os_sleep(100);
    }

    mqtt_report_success();
}
```

这段代码看起来直观，但在真实项目里很危险：

1. 等 ACK 时任务被阻塞，其他事件无法处理。
2. 如果 ACK 永远不来，流程可能永久卡住。
3. 后续加入重试、取消、低功耗、重复命令会很难维护。
4. MQTT、RS485、业务状态混在一起，问题很难定位。

更推荐的做法是：

mqtt_task：只负责 MQTT 收发，把云端命令转换成事件。
app_task：接收事件，更新上下文，驱动业务状态机。
lock_sm：管理唤醒、发送命令、等待 ACK、超时、失败恢复。
rs485_task：只负责 RS485 收发和协议解析。
mqtt_task：只负责发布结果，不直接改业务状态。

一句话理解：

app_task 是“大脑”，service_task 是“手脚”，状态机是“流程记忆”，队列是“传话筒”，事件是“发生了什么”。

1. 先给结论

RTOS 多任务设计，不应该是：

功能来一个，任务开一个。
按键一个任务，显示一个任务，通信一个任务，业务一个任务，上传一个任务……

这种设计前期看起来简单，后期很容易出现：

1. 任务越来越多，优先级越来越乱。
2. 多个任务同时改全局变量。
3. 状态分散在各个任务里，调试困难。
4. 某个任务阻塞，导致业务链路卡住。
5. 队列、事件、状态变量混用，系统行为不可控。
6. 后续加功能时不知道改哪里。

更合理的设计方式是：

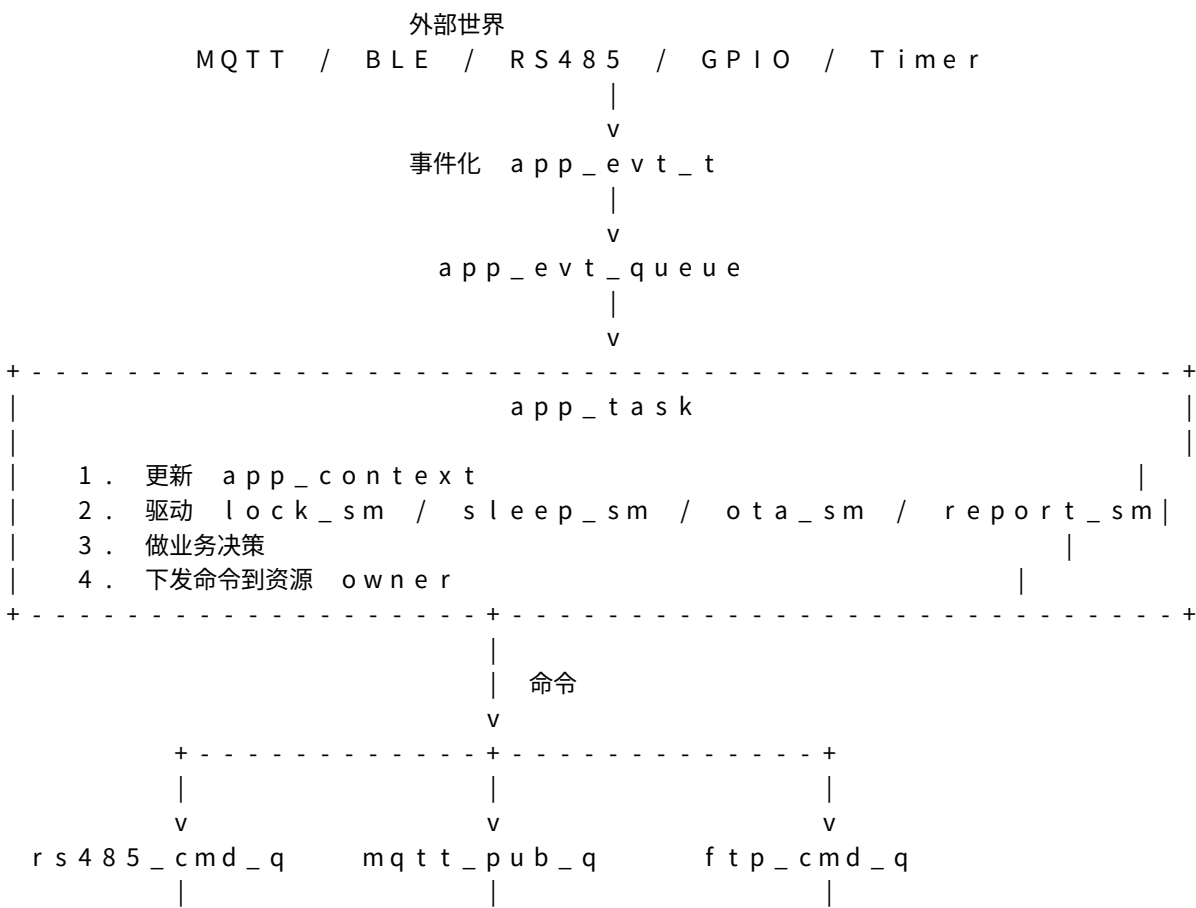
任务：负责并发执行、资源所有权、阻塞隔离。
队列：负责跨任务传递命令或数据。
事件：负责通知系统“发生了什么”。
状态机：负责管理流程阶段、状态转换、超时、重试和异常恢复。

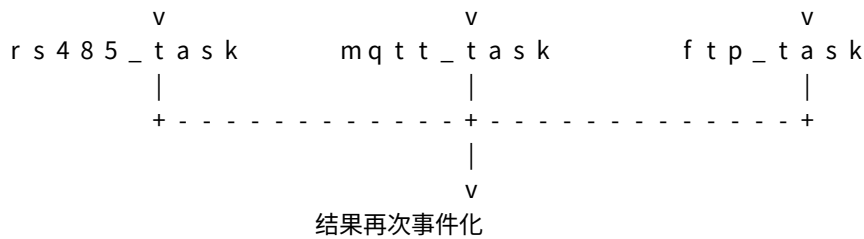
可以先记住这句口诀：

任务看阻塞，队列看跨域；
事件看触发，状态机看流程；
资源有主人，状态有归属；
中断只通知，业务不阻塞；
超时要闭环，失败要恢复。

2 . 总体架构图

推荐的事件流模型如下：





这套架构的核心是：

- 输入统一事件化。
- 业务统一状态机化。
- 输出统一服务化。
- 资源统一 owner 化。
- 异常统一闭环化。
- 日志统一可追踪化。

3 . 四个核心概念

3 . 1 任务：解决“谁独立运行，谁可以阻塞”

任务不是功能模块的代名词。任务是 RTOS 调度的执行上下文，可以理解成“一个能被 RTOS 独立调度的人”。

适合开任务的场景：

- 1 . 这个模块可能长时间阻塞。
- 2 . 这个模块拥有独占资源。
- 3 . 这个模块有独立实时性要求。
- 4 . 这个模块需要和其他逻辑隔离。
- 5 . 这个模块是复杂协议引擎。

典型例子：

- | | |
|-------------|--------|
| RS485 接收解析 | → 适合任务 |
| MQTT 连接和收发 | → 适合任务 |
| FTP 上传 | → 适合任务 |
| 日志落盘 | → 适合任务 |
| GNSS 定位解析 | → 适合任务 |
| OTA 下载 / 写入 | → 适合任务 |

单个开锁动作	→ 不适合单独开任务
单个 G P I O 控制	→ 不适合单独开任务
单个状态判断	→ 不适合单独开任务

一句话判断：

如果这个模块既不会阻塞，也不拥有资源，也没有独立实时性，通常不要单独开任务。

3 . 2 队列：解决“一个任务怎么把消息交给另一个任务”

队列像任务之间的邮箱。一个任务不要直接跑到另一个任务内部做事，而是把请求丢进对方的邮箱。

典型场景：

m q t t _ t a s k 收到云端命令	→ 投递到 a p p _ e v t _ q u e u e
r s 4 8 5 _ t a s k 收到车辆报文	→ 投递到 a p p _ e v t _ q u e u e
a p p _ t a s k 要发属性上报	→ 投递到 m q t t _ p u b _ q u e u e
a p p _ t a s k 要发 R S 4 8 5 控制命令	→ 投递到 r s 4 8 5 _ c m d _ q u e u e
a p p _ t a s k 要上传日志	→ 投递到 f t p _ c m d _ q u e u e
其他模块要写日志	→ 投递到 l o g _ q u e u e

队列适合传递：

- 1 . 命令
- 2 . 事件
- 3 . 数据包
- 4 . 异步请求
- 5 . 执行结果

队列不适合做：

- 1 . 全局状态变量替代品。
- 2 . 高频中断大数据裸搬运。
- 3 . 没有消费者边界的普通函数调用。
- 4 . 用一堆队列绕来绕去模拟状态机。

3 . 3 事件：解决“ 发生了什么 ”

事件强调语义，描述的是已经发生的事实。

例如：

```
收到云端开锁命令  
B L E  已连接  
K L 1 5  电平变化  
R S 4 8 5  收到车辆状态  
R S 4 8 5  收到开锁  A C K  
F T P  上传完成  
O T A  升级失败  
休眠延时超时  
网络已连接  
网络已断开
```

事件一般进入系统主事件队列：

```
a p p _ e v t _ q u e u e
```

由 ` a p p \_ t a s k ` 或系统主状态机统一处理。事件的作用是：

把外部输入、异步结果、超时通知，统一转换成系统可以理解的业务触发条件。

3 . 4 状态机：解决“ 流程现在走到哪一步 ”

状态机像流程进度条，它记住当前走到哪一步，以及收到不同事件后应该跳到哪一步。

只要一个流程具备下面任意特征，就应该考虑状态机：

- 1 . 有多个阶段。
- 2 . 要等待 A C K。
- 3 . 有超时。
- 4 . 有重试。

- 5 . 有成功和失败分支。
- 6 . 会被其他事件打断。
- 7 . 需要异常恢复。

典型适合状态机的流程：

开锁流程
关锁流程
低功耗休眠流程
唤醒流程
OTA 升级流程
FTP 上传流程
MQTT 连接流程
RS485 工厂模式 / EOL 模式 / 正常模式切换
BLE 配对 / 连接 / 控制流程

状态机不一定需要独立任务。很多时候，状态机运行在 `app\_task` 里面：

```
app_task
├── app_sm_dispatch(event)
│   ├── lock_sm_dispatch(event)
│   ├── sleep_sm_dispatch(event)
│   ├── ota_sm_dispatch(event)
│   ├── ftp_sm_dispatch(event)
│   └── report_sm_dispatch(event)
```

4 . 事件、命令、状态不要混在一起

这是初学者最容易混淆的地方。

类型	含义	常见方向	示例
事件	已经发生了什么	driver/service → app_task	`APP_EVT`
命令	希望某个模块去做什么	app_task → service task	`RS485`
状态	当前系统或流程处于什么情况	owner 内部维护	`LOCK_SM_WAIT_ACK`

举例：

APP_EVT_MQTT_CMD_UNLOCK 是事件：
表示 mqtt_task 已经收到云端开锁请求。

RS485_CMD_SEND_UNLOCK 是命令：
表示 app_task 要求 rs485_task 发送开锁帧。

LOCK_SM_WAIT_ACK 是状态：
表示开锁流程已经发出命令，正在等待 ACK。

事件不是命令。事件只描述事实；下一步怎么做，由 `app\_task` 和状态机决定。

5 . 开发前怎么决定 RTOS 框架

拿到一个新项目，不建议一上来就开始写任务。更推荐按下面顺序分析。

第一步：列出系统输入源

先问：系统会被哪些东西触发？

以典型物联网设备 / TBOX / MCU + 通信模块项目为例，输入源可能包括：

- 1 . 云端 MQTT 命令
- 2 . BLE APP 命令
- 3 . RS485 / CAN / UART 总线报文
- 4 . GPIO 中断，例如 KL15、按键、外部唤醒脚
- 5 . 定时器超时
- 6 . 网络状态变化
- 7 . GNSS 定位结果
- 8 . FTP 上传结果
- 9 . OTA 升级结果
- 10 . 文件系统读写结果
- 11 . 电源状态变化
- 12 . 传感器数据变化

可以整理成表格：

输入源	触发条件	产生事件	是否带数据	后续处理者
MQTT	收到开锁命令	`APP_EVT_MQTT_CMD_UNLOCK`	是	app
BLE	APP 连接成功	`APP_EVT_BLE_CONNECTED`	否	app
RS485	收到车辆状态	`APP_EVT_RS485_VEH_STATE`	是	ap
GPIO	KL15 电平变化	`APP_EVT_KL15_CHANGED`	是	app
Timer	ACK 超时	`APP_EVT_UNLOCK_ACK_TIMEOUT`	否	
FTP	上传完成	`APP_EVT_FTP_DONE`	是	app_task

第二步：列出系统输出资源

再问：系统会控制哪些资源？

1. RS485 发送
2. MQTT 发布
3. FTP 上传
4. NAND / Flash 文件写入
5. GPIO 输出控制
6. LED 指示
7. 蜂鸣器
8. GNSS 控制
9. OTA 写入
10. 电源管理控制

这一步的重点是确定：

每个资源由谁拥有？

推荐原则：

一个资源，尽量只有一个 owner。

资源	推荐 owner	其他模块如何使用
RS485 UART	rs485_task	发送 `rs485_cmd_queue`
MQTT Client	mqtt_task	发送 `mqtt_pub_queue`
FTP Client	ftp_task	发送 `ftp_cmd_queue`
NAND 日志文件	log_task	发送 `log_queue`
GNSS	gnss_task	发送 `gnss_cmd_queue`
WAKEUP_OUT GPIO	app/lpm/gpio	选一个 owner 统一接口或命令队

如果多个任务都能直接操作同一个资源，后期非常容易出问题。

第三步：判断哪些模块需要任务

任务应该围绕下面几类对象设计：

1. 资源拥有者

- 2 . 阻塞操作
- 3 . 独立周期任务
- 4 . 高实时处理任务
- 5 . 复杂协议引擎

推荐任务划分示例：

任务	是否必要	主要原因
app_task	通常需要	统一处理系统事件和业务状态机
rs485_task	通常需要	串口接收、协议解析、发送串行化
mqtt_task	通常需要	网络连接、订阅、发布、断线重连
ftp_task	按需	FTP 上传耗时，可能阻塞
log_task	推荐	避免业务任务直接写 Flash / NAND
gnss_task	按需	定位周期、NMEA 解析、AGNSS
ota_task	按需	下载、解压、写入耗时，需要隔离
led_task	不一定	简单 LED 可由 timer / app 控制
key_task	不一定	简单按键可用中断 + 事件
lock_task	不推荐	开锁 / 关锁更适合状态机
sleep_task	不一定	低功耗可由 app 状态机或 lpm_task 管理

第四步：判断哪些流程需要状态机

判断标准很简单：

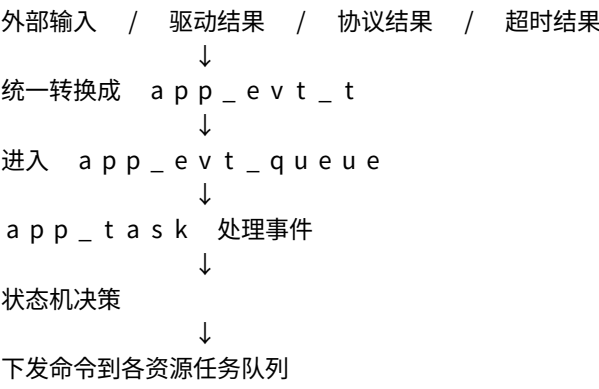
- 是否有阶段？
- 是否要等待？
- 是否有超时？
- 是否有重试？
- 是否会失败？
- 是否可能被打断？

例如开锁流程：



第五步：设计事件流向

推荐方向：



6 . 推荐的目录结构

下面是一套适合中小型嵌入式 RTOS 项目的目录结构：

```
/project
  /app
    app_task.c
    app_event.h
    app_event.c
    app_timer.c
    app_context.h

  /modules
    lock_sm.c
    lock_sm.h
    sleep_sm.c
    sleep_sm.h
    ota_sm.c
    ota_sm.h
    report_sm.c
    report_sm.h

  /services
    rs485_service.c
    rs485_service.h
    mqtt_service.c
    mqtt_service.h
```

```

ftp_service.c
ftp_service.h
log_service.c
log_service.h
gnss_service.c
gnss_service.h

/drivers
uart_drv.c
gpio_drv.c
flash_drv.c
timer_drv.c

/common
ringbuf.c
list.c
crc.c
utils.c

```

分层含义：

`drivers`：

只负责硬件抽象，不理解业务。

`services`：

负责拥有资源，例如 RS485、MQTT、FTP、日志、GNSS。
对外提供异步命令接口。

`modules`：

负责业务状态机，例如开锁、休眠、OTA、上报。
不直接抢占底层资源。

`app`：

系统事件中心，负责事件分发和状态机调度。

7 . 推荐的任务划分

任务	队列	是否允许阻塞	职责
<code>app_task</code>	<code>app_evt_queue</code>	不允许长阻塞	业务决策、状态机调度
<code>rs485_task</code>	<code>rs485_cmd_queue</code>	短阻塞	RS485 收发、协议解析
<code>mqtt_task</code>	<code>mqtt_pub_queue</code>	允许网络等待	MQTT 连接、订阅、发布
<code>ftp_task</code>	<code>ftp_cmd_queue</code>	允许阻塞	FTP 上传/下载
<code>log_task</code>	<code>log_queue</code>	允许文件写	日志落盘
<code>gnss_task</code>	<code>gnss_cmd_queue</code>	允许周期等待	定位、AGNSS、NMEA

重点原则：

`a p p _ t a s k` 不应该做耗时操作。
`a p p _ t a s k` 只做判断、状态切换和命令下发。
 真正耗时的事情交给 `s e r v i c e _ t a s k`。

8 . 核心代码样板

下面代码是框架示意，不依赖具体 `R T O S`。``queue_send``、``queue_recv``、``app_t`
`FreeRTOS`、`RT-Thread`、`Zephyr` 或自研 `R T O S` 的 `A P I`。

8 . 1 应用事件定义

```
typedef enum
{
    APP_EVT_NONE = 0,

    /* 云端命令 */
    APP_EVT_MQTT_CMD_LOCK,
    APP_EVT_MQTT_CMD_UNLOCK,
    APP_EVT_MQTT_CMD_FTP_UPLOAD,
    APP_EVT_MQTT_CMD_OTA,

    /* BLE 事件 */
    APP_EVT_BLE_CONNECTED,
    APP_EVT_BLE_DISCONNECTED,
    APP_EVT_BLE_CMD_LOCK,
    APP_EVT_BLE_CMD_UNLOCK,

    /* RS485 事件 */
    APP_EVT_RS485_VEH_STATE,
    APP_EVT_RS485_LOCK_ACK,
    APP_EVT_RS485_UNLOCK_ACK,
    APP_EVT_RS485_TIMEOUT,

    /* GPIO 事件 */
    APP_EVT_KL15_CHANGED,
    APP_EVT_WAKEUP_PIN_CHANGED,

    /* 定时器事件 */
    APP_EVT_UNLOCK_WAKEUP_DELAY_TIMEOUT,
    APP_EVT_LOCK_ACK_TIMEOUT,
```

```

APP_EVT_UNLOCK_ACK_TIMEOUT,
APP_EVT_SLEEP_DELAY_TIMEOUT,
APP_EVT_PERIOD_REPORT_TIMEOUT,

/* FTP 事件 */
APP_EVT_FTP_DONE,
APP_EVT_FTP_FAIL,

/* OTA 事件 */
APP_EVT_OTA_DONE,
APP_EVT_OTA_FAIL,

/* 网络事件 */
APP_EVT_NET_READY,
APP_EVT_NET_LOST,
APP_EVT_MQTT_CONNECTED,
APP_EVT_MQTT_DISCONNECTED,

} app_evt_id_t;

```

事件数据结构：

```

typedef struct
{
    app_evt_id_t id;
    uint32_t tick;

    union
    {
        struct
        {
            uint8_t lock_state;
            uint8_t power_state;
            uint8_t valid;
        } veh;

        struct
        {
            char path[64];
        } ftp;

        struct
        {
            int code;
        } result;

        struct
        {
            uint8_t level;
        } gpio;
    }
}

```

```

        } data;

    } app_evt_t;

```

设计要点：

1. 所有输入尽量转成 `app_evt_t`。
2. `app_evt_t` 不要太大。
3. 大数据不要直接塞进事件队列。
4. 事件里只放关键状态、索引、短数据。
5. 大文件、大报文用缓冲区或指针引用，但要明确生命周期。

8.2 统一事件投递函数

```

int app_event_post(const app_evt_t *evt)
{
    int ret;

    if (evt == NULL)
    {
        return -1;
    }

    ret = queue_send(app_evt_queue, evt, 0);
    if (ret != 0)
    {
        LOG_E("[APP_EVT] post failed id=%d ret=%d", evt->id, ret);

        /*
         * 关键事件不能静默丢弃。
         * 可以统计丢包次数、触发告警、喂狗保护或进入降级策略。
         */
    }

    return ret;
}

```

不同队列满时的策略可以不同：

`app_evt_queue` 满：严重问题，要报警。
`log_queue` 满：可以丢低等级日志。

`mqtts_pub_queue` 满：普通属性上报可以合并，关键告警不能丢。
`ftp_cmd_queue` 满：通常返回 `busy`。
`rs485_cmd_queue` 满：控制命令要谨慎，可能返回 `busy` 或触发状态机失败。

8.3 app_task 主循环

```
static void app_task(void *arg)
{
    app_evt_t evt;

    app_context_init();
    lock_sm_init();
    sleep_sm_init();
    ota_sm_init();
    report_sm_init();

    while (1)
    {
        if (queue_recv(app_evt_queue, &evt, WAIT_FOREVER)
        {
            app_sm_dispatch(&evt);
        }
    }
}
```

‘`app_task`’ 可以做：

1. 接收系统事件。
2. 更新业务上下文。
3. 驱动状态机。
4. 判断是否需要执行动作。
5. 下发命令到资源任务。

‘`app_task`’ 不应该做：

1. FTP 阻塞上传。
2. 大文件写入。
3. 长时间 `sleep`。
4. `while` 等 ACK。
5. 网络阻塞等待。
6. 复杂串口发送阻塞。

8 . 4 事件分发函数

简单项目可以使用广播式分发：

```
void app_sm_dispatch(const app_evt_t *evt)
{
    if (evt == NULL)
    {
        return;
    }

    app_context_update(evt);

    lock_sm_dispatch(evt);
    sleep_sm_dispatch(evt);
    ota_sm_dispatch(evt);
    report_sm_dispatch(evt);
}
```

实际项目中，如果状态机很多，建议使用路由式分发：

```
void app_sm_dispatch(const app_evt_t *evt)
{
    if (evt == NULL)
    {
        return;
    }

    app_context_update(evt);

    switch (evt->id)
    {
        case APP_EVT_MQTT_CMD_UNLOCK:
        case APP_EVT_BLE_CMD_UNLOCK:
        case APP_EVT_RS485_UNLOCK_ACK:
        case APP_EVT_UNLOCK_ACK_TIMEOUT:
            lock_sm_dispatch(evt);
            break;

        case APP_EVT_KL15_CHANGED:
        case APP_EVT_SLEEP_DELAY_TIMEOUT:
            sleep_sm_dispatch(evt);
            break;

        case APP_EVT_MQTT_CMD_OTA:
```

```

        case APP_EVT_OTA_DONE :
        case APP_EVT_OTA_FAIL :
            ota_sm_dispatch( evt );
            break ;

        default :
            report_sm_dispatch( evt );
            break ;
    }
}

```

总结：

广播式分发简单，路由式分发清晰。
小项目可以广播，大项目建议路由。

8 . 5 开锁状态机示例

先用表格理解状态机四要素：

当前状态	事件	动作	下一个状态
---	---	---	---
IDLE	`APP_EVT_MQTT_CMD_UNLOCK` / `APP_EVT_BLE_CMD_WAKEUP_DEV		
WAKEUP_DEV		`APP_EVT_UNLOCK_WAKEUP_DELAY_TIMEOUT`	
WAIT_ACK			
WAIT_ACK	`APP_EVT_RS485_UNLOCK_ACK`		停止定时器，上报成功
WAIT_ACK	`APP_EVT_UNLOCK_ACK_TIMEOUT`		上报超时失败
WAIT_ACK	再次收到开锁命令	返回 busy 或忽略重复命令	WAIT_ACK

状态定义：

```

typedef enum
{
    LOCK_SM_IDLE = 0,
    LOCK_SM_WAKEUP_DEV,
    LOCK_SM_WAIT_ACK,
} lock_sm_state_t;

static lock_sm_state_t s_lock_state = LOCK_SM_IDLE;

```

状态机处理：

```
void lock_sm_dispatch(const app_evt_t *evt)
{
    switch (s_lock_state)
    {
        case LOCK_SM_IDLE:
            if (evt->id == APP_EVT_MQTT_CMD_UNLOCK ||
                evt->id == APP_EVT_BLE_CMD_UNLOCK)
            {
                s_lock_state = LOCK_SM_WAKEUP_DEV;

                /* 如果 WAKEUP GPIO 归 app_task 管理，可以直接调用；
                 * 如果归 lpm_task/gpio_service 管理，应改为异步命令。 */
                gpio_wakeup_device();

                app_timer_start(TIMER_UNLOCK_WAKEUP_DELAY,
                                0, NULL);
            }
            break;

        case LOCK_SM_WAKEUP_DEV:
            if (evt->id == APP_EVT_UNLOCK_WAKEUP_DELAY_TIMEOUT)
            {
                if (rs485_send_unlock_cmd_async() != 0)
                {
                    mqtt_report_unlock_result_async(LOCK_RET_ERROR,
                                                       s_lock_state);
                    s_lock_state = LOCK_SM_IDLE;
                    break;
                }

                app_timer_start(TIMER_UNLOCK_ACK_TIMEOUT,
                                0, NULL);
                s_lock_state = LOCK_SM_WAIT_ACK;
            }
            break;

        case LOCK_SM_WAIT_ACK:
            if (evt->id == APP_EVT_MQTT_CMD_UNLOCK ||
                evt->id == APP_EVT_BLE_CMD_UNLOCK)
            {
                mqtt_report_unlock_result_async(LOCK_RET_ERROR,
                                                 s_lock_state);
                break;
            }

            if (evt->id == APP_EVT_RS485_UNLOCK_ACK)
            {
                app_timer_stop(TIMER_UNLOCK_ACK_TIMEOUT);
                mqtt_report_unlock_result_async(LOCK_RET_SUCCESS,
                                                 s_lock_state);
                s_lock_state = LOCK_SM_IDLE;
            }
            else if (evt->id == APP_EVT_UNLOCK_ACK_TIMEOUT)
            {
                mqtt_report_unlock_result_async(LOCK_RET_ERROR,
                                                 s_lock_state);
                s_lock_state = LOCK_SM_IDLE;
            }
    }
}
```

```

        break;

    default:
        s_lock_state = LOCK_SM_IDLE;
        break;
}
}

```

这个状态机的关键点：

1. 收到命令后不阻塞。
2. 唤醒稳定延时通过 `timer` 事件完成。
3. `ACK` 等待通过状态机处理。
4. 超时也是事件。
5. 成功和失败都有出口。
6. 重复命令有 `busy` 处理。
7. 状态最终回到 `IDLE`。

8.6 RS485 服务任务示例

RS485 命令定义：

```

typedef enum
{
    RS485_CMD_SEND_LOCK,
    RS485_CMD_SEND_UNLOCK,
    RS485_CMD_SEND_QUERY_STATE,
} rs485_cmd_id_t;

typedef struct
{
    rs485_cmd_id_t id;
    uint8_t data[32];
    uint16_t len;
} rs485_cmd_t;

```

异步发送接口：

```

int rs485_send_unlock_cmd_async(void)
{
    rs485_cmd_t cmd;

```

```

    memset(&cmd, 0, sizeof(cmd));
    cmd.id = RS485_CMD_SEND_UNLOCK;

    return queue_send(rs485_cmd_queue, &cmd, 0);
}

```

RS485 任务：

```

static void rs485_task(void *arg)
{
    rs485_cmd_t cmd;

    rs485_hw_init();

    while (1)
    {
        if (queue_recv(rs485_cmd_queue, &cmd, 10) == 0)
        {
            rs485_handle_cmd(&cmd);
        }

        rs485_rx_process();
    }
}

```

收到 ACK 后转成 app 事件：

```

static void rs485_on_unlock_ack(void)
{
    app_evt_t evt;

    memset(&evt, 0, sizeof(evt));
    evt.id = APP_EVT_RS485_UNLOCK_ACK;
    evt.tick = os_tick_get();

    app_event_post(&evt);
}

```

重点：

rs485_task 负责 RS485 收发和协议解析。
 它不应该自己决定“开锁成功后业务怎么走”。
 它只需要把 ACK 转成 APP_EVT_RS485_UNLOCK_ACK。
 真正的业务决策交给 app_task / lock_sm。

8.7 MQTT 服务任务示例

MQTT 发布命令：

```
typedef enum
{
    MQTT_PUB_PROPERTY,
    MQTT_PUB_EVENT,
    MQTT_PUB_LOG,
} mqtt_pub_type_t;

typedef struct
{
    mqtt_pub_type_t type;
    char topic[128];
    char payload[256];
    uint8_t qos;
} mqtt_pub_msg_t;
```

发布接口：

```
int mqtt_publish_async(const char *topic, const char *payload,
                        mqtt_pub_type_t type, uint8_t qos)
{
    mqtt_pub_msg_t msg;

    if (topic == NULL || payload == NULL)
    {
        return -1;
    }

    memset(&msg, 0, sizeof(msg));
    strncpy(msg.topic, topic, sizeof(msg.topic) - 1);
    strncpy(msg.payload, payload, sizeof(msg.payload) - 1);
    msg.qos = qos;

    return queue_send(mqtt_pub_queue, &msg, 0);
}
```

MQTT 任务：

```

static void mqtt_task(void *arg)
{
    mqtt_pub_msg_t msg;

    mqtt_client_init();

    while (1)
    {
        mqtt_keepalive_process();
        mqtt_recv_process();

        if (queue_recv(mqtt_pub_queue, &msg, 100) == 0)
        {
            mqtt_client_publish(msg.topic, msg.payload, ms
        }
    }
}

```

收到云端命令后投递 app 事件：

```

static void mqtt_on_unlock_cmd(void)
{
    app_evt_t evt;

    memset(&evt, 0, sizeof(evt));
    evt.id = APP_EVT_MQTT_CMD_UNLOCK;
    evt.tick = os_tick_get();

    app_event_post(&evt);
}

```

重点：

`mqtt_task` 不应该直接执行开锁逻辑。
它只负责把云端命令转换成事件。

9 . 一个完整链路：云端开锁怎么走

- 1 . 云端下发开锁命令。
- 2 . `mqtt_task` 收到命令。
- 3 . `mqtt_task` 解析后投递 `APP_EVT_MQTT_CMD_UNLOCK`。

4. `app_task` 从 `app_evt_queue` 取出事件。
5. `lock_sm` 进入 `WAKEUP_DEV` 状态。
6. `lock_sm` 唤醒外设，并启动唤醒稳定定时器。
7. 定时器产生 `APP_EVT_UNLOCK_WAKEUP_DELAY_TIMEOUT`。
8. `lock_sm` 下发 `rs485_send_unlock_cmd_async()`。
9. `rs485_task` 从 `rs485_cmd_queue` 取出命令并发送。
10. `rs485_task` 收到 `ACK`。
11. `rs485_task` 投递 `APP_EVT_RS485_UNLOCK_ACK`。
12. `lock_sm` 收到 `ACK` 事件。
13. `lock_sm` 触发 `mqtt_report_unlock_result_async()`。
14. `mqtt_task` 从 `mqtt_pub_queue` 取出消息并发布。
15. `lock_sm` 回到 `IDLE`。

这个链路的好处：

1. `MQTT` 只负责收发，不负责业务决策。
2. `RS485` 只负责协议，不负责业务状态。
3. `app_task` 只负责决策，不做阻塞发送。
4. 状态机负责流程阶段和异常处理。
5. 每一步都有明确 `owner`。
6. 超时和失败都有闭环。

10. `app_context` 应该放什么

``app_context`` 是 ``app_task`` 维护的系统上下文，保存业务决策需要的基础状态。

适合放：

网络是否可用
`MQTT` 是否连接
 车辆当前锁状态
 车辆当前电源状态
`KL15` 当前电平
`BLE` 是否连接
 当前 `OTA` 状态摘要
 当前 `FTP` 是否 `busy`

不适合放：

大文件内容
 完整 `MQTT payload`

完整 RS485 原始帧缓存
服务任务内部私有状态
临时流程状态

注意区分“系统事实状态”和“流程状态”：

```
vehicle_lock_state = LOCKED
```

表示车辆现在是锁住的。

```
lock_sm_state = LOCK_SM_WAIT_ACK
```

表示开锁流程正在等待 ACK。

`app\_context` 不是全局变量垃圾桶。只有明确 owner 才能更新对应字段。

1.1 事件数据生命周期

跨任务传指针时，必须回答三个问题：

1. 这块内存谁申请？
2. 谁释放？
3. 消费者处理前，它会不会被覆盖？

反例：

```
void mqtt_on_msg(char *payload)
{
    app_evt_t evt;

    evt.id = APP_EVT_MQTT_CMD_UNLOCK;
    evt.data.ptr = payload;

    app_event_post(&evt);
}
```

如果 `payload` 是 MQTT 接收缓冲区，`mqtt\_on\_msg` 返回后缓冲区可能被复用。`
理时，指针可能已经失效。

推荐规则：

小数据：直接拷贝进事件。
大数据：使用内存池、引用计数、固定缓冲区索引。
禁止把临时栈变量地址投递到队列。

1 2 . 优先级怎么设计

R T O S 任务优先级不是越高越好。

基本原则：

高优先级任务必须短小，不能长时间阻塞。
低优先级任务可以做耗时但不紧急的事情。

推荐思路：

- 中断
- > 高实时接收任务
 - > 协议处理任务
 - > a p p 业务任务
 - > 网络 / 文件 / 日志任务

任务	优先级建议	原因
U A R T R X / D M A 处理	高	防止数据丢失
r s 4 8 5 _ t a s k	较高	协议收发有时序要求
a p p _ t a s k	中	业务事件调度中心
m q t t _ t a s k	中	网络收发、保活
g n s s _ t a s k	中低	周期定位
f t p _ t a s k	中低	上传耗时，但不应影响核心控制
l o g _ t a s k	低	日志落盘不能影响业务

高优先级任务不要做：

- 1 . 写 F l a s h 。
- 2 . 做 F T P / H T T P 。
- 3 . 大量打印日志。
- 4 . 长时间拿锁。

5 . 等待低优先级任务释放资源。

否则可能出现优先级反转、任务饿死、串口丢包、看门狗复位。

1 3 . 队列大小怎么定

队列不是越大越好。队列太小会丢消息，太大会掩盖系统处理不过来的问题。

设计队列大小时要考虑：

- 1 . 生产者最坏突发速度。
- 2 . 消费者最慢处理速度。
- 3 . 单条消息大小。
- 4 . 是否允许丢弃。
- 5 . 是否需要背压机制。

队列	建议特点
app_evt_queue	中等大小，不能轻易丢关键事件
rs485_cmd_queue	不宜太大，控制命令应有顺序和超时
mqtt_pub_queue	可以稍大，但要处理网络断开堆积
log_queue	可以较大，但允许低优先级日志丢弃
ftp_cmd_queue	通常很小，甚至只允许一个任务
ota_cmd_queue	通常很小，避免并发 OTA

常见策略：

- 关键控制事件：不能丢，满了要报警或触发降级。
- 普通状态上报：可以合并或覆盖。
- 调试日志：可以丢弃低等级日志。
- 周期数据：可以保留最新值，丢旧值。

1 4 . 中断里应该做什么

中断里只做最小事情：

- 1 . 清中断标志。
- 2 . 读取必要寄存器或 G P I O 电平。
- 3 . 保存最小数据。
- 4 . 投递事件或释放信号量。
- 5 . 尽快退出。

不应该在中断里做：

- 1 . J S O N 解析。
- 2 . 文件写入。
- 3 . M Q T T 发布。
- 4 . F T P 上传。
- 5 . 大量日志打印。
- 6 . 复杂状态机切换。
- 7 . 长时间循环等待。

G P I O 中断示例：

```
static void gpio_irq_handler(void)
{
    uint8_t level;

    level = gpio_read(KL15_PIN);

    app_event_post_gpio_from_isr(APP_EVT_KL15_CHANGED, level);
}
```

这样设计的好处：

- 1 . 中断响应快。
- 2 . 业务逻辑在任务上下文运行。
- 3 . 可以打印日志。
- 4 . 可以启动定时器。
- 5 . 可以做状态机判断。
- 6 . 系统更容易调试。

1 5 . 超时一定要事件化

不要这样写：

```
send_cmd();

while (!ack)
{
    os_sleep(100);
}
```

更好的方式是：

1. 发送命令。
2. 状态机进入 WAIT_ACK。
3. 启动 ACK 超时定时器。
4. 收到 ACK → 投递 ACK 事件。
5. 定时器超时 → 投递 TIMEOUT 事件。
6. 状态机根据事件决定成功、失败或重试。

伪代码：

```
case SM_SEND_CMD:
    send_cmd_async();
    app_timer_start(TIMER_ACK_TIMEOUT, 3000);
    sm_state = SM_WAIT_ACK;
    break;

case SM_WAIT_ACK:
    if (evt->id == APP_EVT_ACK)
    {
        app_timer_stop(TIMER_ACK_TIMEOUT);
        sm_state = SM_DONE;
    }
    else if (evt->id == APP_EVT_ACK_TIMEOUT)
    {
        sm_state = SM_FAIL;
    }
    break;
```

几乎所有带 ACK 的协议流程都可以套用这个模式。

16. 信号量、事件组、队列怎么选

机制	适合场景	不适合场景
Queue	传递带数据的消息、命令、事件	高频大数据裸搬运
Semaphore	通知某个任务有资源或动作完成	表达复杂业务事件
Event Flag / Event Group	多个 bit 条件组合等待	传递复杂参数
Mutex	保护共享资源	做任务通知
State Machine	管理多阶段流程	简单一次性动作

例子：

```

UART DMA 收到一帧数据：
可以用 semaphore 唤醒 rs485_task；
具体数据放 ring buffer；
解析出 ACK 后再投递 app_evt_t。

```

1 7 . 低功耗流程尤其需要状态机

低功耗不是简单调用 `enter_sleep()`。进入休眠前通常要确认：

- 1 . 没有正在进行的开锁 / 关锁流程。
- 2 . 没有待发送的关键 MQTT 消息。
- 3 . 日志是否需要 flush。
- 4 . RS485 是否空闲。
- 5 . FTP / OTA 是否正在进行。
- 6 . 唤醒源是否配置完成。
- 7 . 外设是否已经关闭。

因此低功耗流程通常适合 `sleep_sm`，而不是在 `lpm_task` 里到处写 `if` 判断。

1 8 . 常见反模式

1 8 . 1 一个功能一个任务

错误做法：

```
lock_task
unlock_task
sleep_task
report_task
```

问题：

- 1 . 业务状态分散。
- 2 . 任务之间互相依赖。
- 3 . 优先级难设计。
- 4 . 全局变量容易乱。
- 5 . 后期不好维护。

更推荐：

```
app_task + lock_sm + sleep_sm + report_sm
```

18.2 多个任务直接改同一个状态

错误做法：

```
/* mqtt_task */
g_sleep_state = 0;

/* ble_task */
g_sleep_state = 0;

/* rs485_task */
g_sleep_state = 1;

/* lpm_task */
if (g_sleep_state)
{
    enter_sleep();
}
```

问题：

状态 `owner` 不明确。
谁改的？为什么改？什么时候改？很难追踪。

更推荐：

所有任务投递事件。
`app_task / sleep_sm` 统一修改 `sleep_state`。

18.3 `app_task` 做耗时操作

错误做法：

```
case APP_EVT_FTP_UPLOAD:
    ftp_upload_file();          /* 阻塞很久 */
    break;
```

问题：

`app_task` 被卡住，其他事件无法处理。

更推荐：

```
case APP_EVT_FTP_UPLOAD:
    ftp_upload_async(&req);
    break;
```

FTP 完成后再投递：

```
APP_EVT_FTP_DONE
APP_EVT_FTP_FAIL
```

18.4 while 等 ACK

错误做法：

```
send_cmd();
while (!ack)
{
    os_sleep(100);
}
```

更推荐：

状态机 + ACK 事件 + 超时事件

18.5 中断里做业务

错误做法：

```
void gpio_irq_handler(void)
{
    parse_json();
    mqtt_publish();
    file_write();
}
```

更推荐：

中断只投递事件，业务放到任务上下文处理。

19. RTOS API 映射表

概念	FreeRTOS	RT-Thread	Zephyr
Task	<code>`xTaskCreate`</code>	<code>`rt_thread_create`</code>	<code>`k_thread_create`</code>
Queue	<code>`xQueueCreate`</code>	<code>`rt_mq_create`</code>	<code>`k_msgq_create`</code>
Semaphore	<code>`xSemaphoreCreateBinary`</code>	<code>`rt_sem_create`</code>	<code>`k_semaphore_create`</code>
Mutex	<code>`xSemaphoreCreateMutex`</code>	<code>`rt_mutex_create`</code>	<code>`k_mutex_create`</code>
Timer	<code>`xTimerCreate`</code>	<code>`rt_timer_create`</code>	<code>`k_timer_create`</code>
Event Flag	<code>`xEventGroupCreate`</code>	<code>`rt_event_create`</code>	<code>`k_event_create`</code>

2 0 . 设计检查表

2 0 . 1 任务检查

- ☐ 每个任务是否有明确职责？
- ☐ 是否存在“为了功能而开任务”的情况？
- ☐ 高优先级任务是否足够短？
- ☐ 是否有任务长时间阻塞？
- ☐ 阻塞任务是否会影响核心业务？
- ☐ 每个任务栈大小是否评估过？
- ☐ 是否有任务可能饿死？

2 0 . 2 队列检查

- ☐ 队列生产者和消费者是否明确？
- ☐ 队列满了怎么处理？
- ☐ 关键消息是否允许丢？
- ☐ 普通消息是否可以合并？
- ☐ 队列元素是否过大？
- ☐ 队列是否可能造成内存浪费？

2 0 . 3 事件检查

- ☐ 所有外部输入是否事件化？
- ☐ 超时是否事件化？
- ☐ ACK 是否事件化？
- ☐ 失败结果是否事件化？
- ☐ 事件 ID 是否有清晰语义？
- ☐ 事件数据生命周期是否安全？

2 0 . 4 状态机检查

- [] 每个状态是否有明确含义？
- [] 每个状态是否有出口？
- [] 等待状态是否有超时？
- [] 失败后是否有恢复路径？
- [] 是否支持重复命令或异常命令？
- [] 是否记录状态切换日志？

2 0 . 5 资源检查

- [] 每个硬件资源是否有唯一 owner？
- [] 多任务访问共享资源是否有锁？
- [] 是否存在多个任务直接操作同一外设？
- [] 文件系统访问是否可能冲突？
- [] 网络客户端是否被多个任务直接调用？
- [] G P I O 控制权是否清晰？

2 0 . 6 调试检查

- [] 能否看到事件投递日志？
- [] 能否看到队列满日志？
- [] 能否看到状态切换日志？
- [] 能否看到超时日志？
- [] 能否区分收到命令、执行成功、A C K 成功、上报成功？
- [] 能否定位失败发生在哪一层？

2 1 . 调试五问

系统出问题时，沿着链路问：

- 1 . 事件有没有投递成功？
- 2 . a p p _ t a s k 有没有收到事件？
- 3 . 状态机有没有切换？
- 4 . 命令有没有发到资源队列？
- 5 . 资源任务有没有执行并返回结果？

建议日志点：

问题	日志位置
事件有没有投递成功	`app_event_post`
app_task 有没有收到	`app_task` 主循环
状态机有没有切换	`xxx_sm_dispatch`
命令有没有发出	`xxx_async`
服务任务有没有执行	`xxx_task`
结果有没有回来	service 回调 / `app_event_post`

对状态机来说，最重要的日志是：

事件 + 旧状态 + 新状态 + 动作 + 结果

示例：

```
LOG_I (" [LOCK_SM] evt=%d state:%d->%d", evt->id, old_state, new_state);
LOG_I (" [RS485] send unlock cmd ret=%d", ret);
LOG_I (" [APP_EVT] post id=%d ret=%d", evt.id, ret);
LOG_W (" [LOCK_SM] wait ack timeout");
LOG_E (" [MQTT] publish failed ret=%d", ret);
```

2.2 . 小项目和中型项目不要套同一套复杂度

如果项目很小，不需要机械套用完整框架。

最小版本可以是：

```
app_task
rs485_task
app_evt_queue
rs485_cmd_queue
lock_sm
```

适合：

单 MCU
单串口
简单控制流程

标准版本可以是：

app_task
rs485_task
mqtt_task
log_task
ftp_task
ota_task
gnss_task
多个状态机

适合：

TBOX
网关
带云端、OTA、日志、低功耗的设备

架构设计的目标不是复杂，而是边界清晰。

23 . 最终总结

RTOS 架构设计的核心不是多任务，而是边界：

任务：谁独立执行，谁可以阻塞。
队列：谁给谁传消息。
事件：系统发生了什么。
状态机：流程走到哪一步。
owner：资源和状态归谁管。
超时：失败以后怎么回来。
日志：出问题时怎么定位。

一个好的 RTOS 设计应该做到：

输入统一事件化；
业务统一状态机化；
输出统一服务化；
资源统一 owner 化；
异常统一闭环化；
日志统一可追踪化。

最后再用一句话概括：

R T O S 项目真正优秀的设计，不是任务多，也不是队列多，
而是每个任务、每个队列、每个事件、每个状态机都有明确边界。
系统出了问题时，你能快速判断：
事件有没有来？状态有没有变？命令有没有发？A C K 有没有回？
资源任务有没有执行？失败在哪一层？